

TITLE: DEVELOPING AND DEPLOYING A SIMPLE CHAINCODE

1. AIM

To develop, package, install, approve, commit, and execute a simple asset transfer chaincode in a Hyperledger Fabric test network using the JavaScript programming language.

2. TOOLS / SOFTWARE REQUIRED

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images

3. HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

4. PROCEDURE

Step 1: Navigate to the Test Network

Open the terminal and move to the Hyperledger Fabric test network directory.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

Start the blockchain network with Certificate Authorities.

```
./network.sh up createChannel -ca
```

Step 3: Deploy the JavaScript Chaincode

Deploy the Asset Transfer Basic chaincode.

```
./network.sh deployCC  
-ccl javascript  
-ccn basic  
-ccp ../asset-transfer-basic/chaincode-javascript
```

Step 4: Verify Installed Chaincode

Check whether the chaincode is installed.

```
peer lifecycle chaincode queryinstalled
```

Step 5: Invoke the Chaincode

Initialize the ledger.

```
peer chaincode invoke  
-o localhost:7050  
--ordererTLSHostnameOverride orderer.example.com  
--tls  
--cafile $ORDERER_CA  
-C mychannel  
-n basic  
-c '{"function":"InitLedger","Args":[]}'
```

Step 6: Query the Ledger

Retrieve all assets stored in the blockchain.

```
peer chaincode query  
-C mychannel  
-n basic  
-c '{"Args":["GetAllAssets"]}'
```

Step 7: Create a New Asset

```
peer chaincode invoke  
-o localhost:7050  
--ordererTLSHostnameOverride orderer.example.com  
--tls  
--cafile $ORDERER_CA  
-C mychannel  
-n basic  
-c '{"function":"CreateAsset","Args":["asset10","Black","15","Arun","800"]}'
```

Step 8: Query the Newly Created Asset

```
peer chaincode query  
-C mychannel  
-n basic  
-c '{"Args":["ReadAsset","asset10"]}'
```

Step 9: Transfer Asset Ownership

```
peer chaincode invoke
-o localhost:7050
--ordererTLSHostnameOverride orderer.example.com
--tls
--cafile $ORDERER_CA
-C mychannel
-n basic
-c '{"function":"TransferAsset","Args":["asset10","David"]}'
```

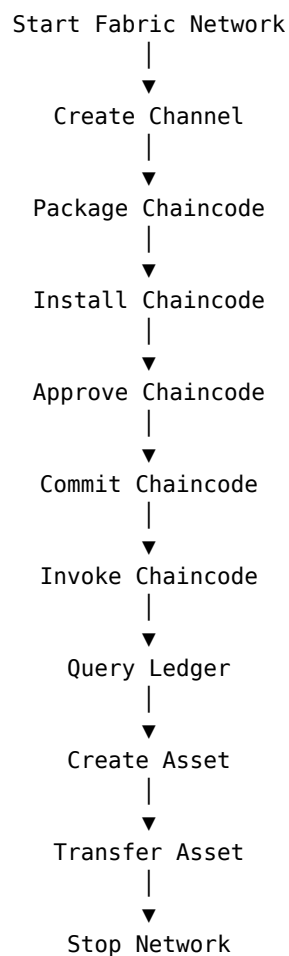
Step 10: Verify Updated Asset

```
peer chaincode query
-C mychannel
-n basic
-c '{"Args":["ReadAsset","asset10"]}'
```

Step 11: Stop the Network

```
./network.sh down
```

5. FLOW OF PROCESS



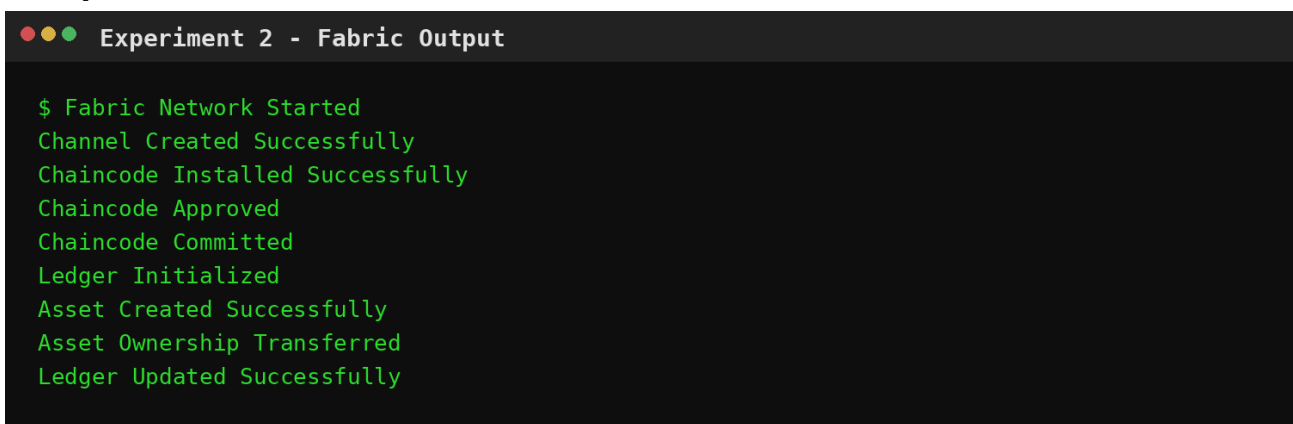
6. SUMMARY OF KEY COMMANDS

Purpose	Command
Navigate	<code>cd ~/fabric-dev/fabric-samples/test-network</code>
Start network	<code>./network.sh up createChannel -ca</code>
Deploy chaincode	<code>./network.sh deployCC</code>
Query ledger	<code>peer lifecycle chaincode queryinstalled</code>
Invoke transaction	<code>peer chaincode invoke</code>
Query ledger	<code>peer chaincode query</code>
Stop network	<code>./network.sh down</code>

7. OUTPUT

The terminal output below is rendered from the output blocks supplied in the source experiment.

Output 1



```

Experiment 2 - Fabric Output

$ Fabric Network Started
Channel Created Successfully
Chaincode Installed Successfully
Chaincode Approved
Chaincode Committed
Ledger Initialized
Asset Created Successfully
Asset Ownership Transferred
Ledger Updated Successfully

```

Fig. 2.1 – Fabric output corresponding to the source experiment output.

8. RESULT

The simple JavaScript chaincode was successfully developed, deployed, installed, approved, committed, and executed on the Hyperledger Fabric test network. Asset records were created, queried, and transferred successfully, demonstrating the basic lifecycle of smart contracts in Hyperledger Fabric.